

Температура изменяется от $-34,88$ до $31,94$ °C; медиана равна $19,08$ °C. Для precipitation_mm медиана составляет $0,04$ мм при максимуме $25,85$ мм, а для wind_speed_ms медиана $0,25$ м/с при максимуме $7,42$ м/с. Такие соотношения уже указывают на асимметричность некоторых распределений и необходимость визуальной проверки.

6. Типы данных и предобработка

Предобработка — это не обязательное исправление каждого столбца, а приведение таблицы к типам и представлениям, соответствующим смыслу данных. Категории целесообразно хранить как category, небольшие целочисленные коды — в компактных целых типах, числовые преобразования выполнять векторно. Удалять строки следует только при наличии обоснованной причины.

В данном файле пропусков и полных дубликатов нет, поэтому очистка не должна искусственно удалять наблюдения. Вместо этого полезно оптимизировать dtypes и создать вычисляемые признаки, не меняя исходную информацию.

Практический блок 6.1. Оптимизация типов

Сначала будет измерен объём исходного DataFrame в памяти. Затем создаётся clean_df, в котором два столбца получают типы, соответствующие их смыслу и диапазону значений.

После преобразования память измеряется повторно, а код вычисляет относительное снижение. Такой эксперимент показывает, что предобработка включает не только удаление ошибок, но и приведение представления данных к более эффективной схеме без потери информации.

Метод memory_usage(deep=True) возвращает оценку памяти по компонентам DataFrame, а sum() суммирует её до общего объёма. df.copy() создаёт независимую копию таблицы, чтобы не изменять исходный объект.

Метод astype() явно преобразует dtype. Тип category хранит повторяющиеся текстовые категории через компактные коды, а int8 использует 8-битное целое для небольших числовых кодов. f-строки позволяют вставлять вычисляемые выражения в текст. Спецификаторы :.2f и :.1f задают число знаков после точки.

```
before = df.memory_usage(deep=True).sum()

clean_df = df.copy()
clean_df["type_name"] = clean_df["type_name"].astype("category")
clean_df["type_id"] = clean_df["type_id"].astype("int8")
```

```

after = clean_df.memory_usage(deep=True).sum()
print(f"До: {before / 1024**2:.2f} MiB")
print(f"После: {after / 1024**2:.2f} MiB")
print(f"Снижение: {(1 - after / before) * 100:.1f}%")

```

Результат выполнения

```

До: 3.18 MiB
После: 1.48 MiB
Снижение: 53.5%

```

Преобразование `type_name` в `category` особенно эффективно, потому что вместо 26 681 повторяющейся строки Pandas хранит компактные коды категорий и таблицу уникальных значений.

Практический блок 6.2. Создание вычисляемых признаков

В таблицу будут добавлены два разных по смыслу производных признака. `wind_speed_kmh` является преобразованием единиц измерения и не добавляет новой физической информации, тогда как `is_forest_fire` кодирует принадлежность строки к одной категории в бинарном виде.

После создания столбцов выводятся исходная скорость, преобразованная скорость и логический индикатор, чтобы визуально проверить корректность вычислений на первых наблюдениях.

Присваивание вида `clean_df["new_column"] = expression` создаёт новый столбец либо заменяет существующий.

Арифметика над `Series` выполняется векторно. Умножение `wind_speed_ms` на 3.6 применяется ко всем строкам, после чего `round(2)` округляет значения.

Сравнение `Series` со строкой "Forest fire" также выполняется поэлементно и возвращает логическую `Series` из `True/False`.

Выражение `clean_df[...].head()` выбирает несколько полей и показывает первые пять строк результата.

```

clean_df["wind_speed_kmh"] = (
    clean_df["wind_speed_ms"] * 3.6
).round(2)

clean_df["is_forest_fire"] = (
    clean_df["type_name"] == "Forest fire"
)

print(clean_df[
    ["type_name", "wind_speed_ms", "wind_speed_kmh",
    "is_forest_fire"]
].head())

```

Таблица 6. Результат выполнения блока 6.2. в виде таблицы

type_name	wind_speed_ms	wind_speed_kmh	is_forest_fire
Natural fire	1.14	4.10	False
Natural fire	1.14	4.10	False
Forest fire	0.80	2.88	True
Natural fire	0.97	3.49	False
Natural fire	1.14	4.10	False

wind_speed_kmh — линейное преобразование единиц, а is_forest_fire — логический индикатор категории. Первый столбец не добавляет новой физической информации, но удобен для представления. Второй пригоден для фильтрации и может выступать бинарной меткой в последующих задачах.

7. Фильтрация, группировка и агрегирование

Фильтрация отвечает на вопрос «какие строки удовлетворяют условию?», а группировка — «как отличается агрегат между категориями?». В Pandas булево выражение формирует маску длины N, после чего loc выбирает строки и столбцы. Метод groupby() делит данные на группы, а agg() вычисляет несколько статистик за один проход логической операции.

Практический блок 7.1. Распределение наблюдений по типам

В следующем фрагменте будет построена таблица распределения классов: для каждого type_name определяются и число наблюдений, и его доля в процентах. Результат нужно интерпретировать не только как рейтинг частот, но и как диагностику баланса выборки.

Метод value_counts() подсчитывает частоту каждого уникального значения Series и по умолчанию сортирует категории по убыванию частоты.

to_frame("n") преобразует полученную Series в DataFrame со столбцом n.

Метод assign() добавляет вычисляемый столбец и возвращает новый DataFrame; в данном случае значение share_pct задаётся через lambda — небольшую анонимную функцию.

len(df) возвращает общее число строк, поэтому $n / \text{len}(\text{df}) * 100$ переводит абсолютные частоты в проценты.

```
type_stats = (
    df["type_name"]
    .value_counts()
    .to_frame("n")
    .assign(share_pct=lambda x: x["n"] / len(df) * 100)
```